OperPan

Plan de Migración

Versión: 2

Fecha: 16/10/2026

Control de Cambios

Versión	Causa del Cambio	Responsable del Cambio	Fecha del Cambio
1	Versión inicial	Santiago Muñeton Hernandez y Paula Andrea Herrera Carrillo	15/10/2026
2	Nueva estructura	Santiago Muñeton Hernandez y Paula Andrea Herrera Carrillo	16/10/2026

Tabla de contenido

Introducción 3
Objeto 4
Alcance 5
Razones para la Migración de Datos 5
Base de Datos Origen y Posible Base de Datos Destino 6
Tipo de Migración 8
Metodología de la Migración de Datos 9
- Planeación 9
- Migración 12
- Validación de datos 14
- Cierre del proyecto 15

Técnica de Migración de Datos 16
- Extracción 16
- Diccionario de Datos de la Base de Datos Origen 17
- Mapeo de Campos 17
- Transformación 19
- Carga 19
- Validación 20

Estrategia de Migración de Datos 21
Lista de Referencias 22

Introducción

El presente documento describe el Plan de Migración de la base de datos del proyecto OperPan, sistema de gestión interna desarrollado en Django para la panadería y cafetería Estación Paisa. El plan tiene como finalidad trasladar la base de datos desde un entorno local (XAMPP con MySQL) hacia un servidor de hosting en la nube (PythonAnywhere), garantizando la integridad, consistencia y disponibilidad de la información que soporta los módulos de usuarios, asistencia, horarios, tareas, novedades y memorandos.

La migración se plantea como homogénea, ya que se conserva el mismo motor de base de datos MySQL en ambos entornos. Esto simplifica la transferencia al no requerir conversión de tipos de datos ni adaptación de sintaxis SQL, y permite mantener el funcionamiento nativo del ORM de Django sin modificaciones adicionales.

El documento se estructura siguiendo las buenas prácticas de planeación de migraciones: describe el origen y destino, justifica las razones del cambio, define el tipo, la metodología, la técnica y la estrategia de migración, e incluye los criterios de validación que servirán como evidencia para autorizar el paso al entorno de producción.

Objeto

El objeto de este plan es definir el procedimiento técnico, ordenado y verificable para migrar la base de datos de OperPan desde el entorno de desarrollo local hacia el servidor de hosting PythonAnywhere, estableciendo las actividades, los recursos, los riesgos y los criterios de aceptación necesarios. Con ello se busca asegurar que la información almacenada se traslade de forma íntegra, consistente y disponible para el uso continuo de la aplicación en producción.

Alcance

Este plan está dirigido al equipo de desarrollo y aseguramiento de la calidad (QA) del proyecto OperPan, así como al instructor encargado de la evaluación académica. El alcance comprende:

La evaluación de PythonAnywhere como alternativa de alojamiento para Django y MySQL.

La identificación del origen (XAMPP con MySQL) y del destino propuesto (PythonAnywhere con MySQL).

La definición del tipo, la técnica y la estrategia de migración.

La descripción del procedimiento por fases: planeación, migración, validación y cierre.

El establecimiento de criterios de validación técnica y funcional que permitan verificar la integridad de los datos y la operatividad de la aplicación.

Quedan fuera del alcance la migración a motores distintos a MySQL (como SQL Server o PostgreSQL), la conversión de esquemas heterogéneos y la adaptación del código fuente a controladores diferentes al actualmente configurado en Django.

Razones para la Migración de Datos

Las razones que justifican la migración de la base de datos de OperPan son las siguientes:

Accesibilidad: permitir que los usuarios de Estación Paisa accedan a la información desde cualquier dispositivo con conexión a Internet, sin depender del equipo local donde actualmente se ejecuta XAMPP.

Disponibilidad de la información: mantener la base de datos operativa de forma continua para el funcionamiento de los módulos de usuarios, asistencia, horarios, tareas, novedades y memorandos.

Integración con Django: es necesario trasladar la base de datos para establecer la conexión entre la aplicación Django (desplegada en PythonAnywhere) y el servidor de base de datos en la nube.

Funcionamiento en línea: permitir que OperPan pase de un entorno local a un entorno web, haciendo posible su uso desde Internet y facilitando su evaluación académica.

Respaldo y persistencia: aprovechar los mecanismos de copia de seguridad y persistencia que ofrece un entorno de hosting administrado.

Base de Datos Origen y Posible Base de Datos Destino

Base de datos origen:

Elemento	Descripción
Sistema Gestor de Base de Datos	MySQL (provisto por XAMPP)
Herramienta de administración	phpMyAdmin / MySQL Workbench
Entorno de ejecución	Local (127.0.0.1)
Nombre del esquema	operpan
Cotejamiento (Collation)	utf8mb4_general_ci

Estructura	13 tablas principales que soportan los módulos del sistema

Base de datos destino propuesta:

Elemento	Descripción
Proveedor de hosting	PythonAnywhere
Sistema Gestor de Base de Datos	MySQL (plan de pago o complemento de base de datos)
Entorno de ejecución	Producción (nube)
Nombre del esquema	Definido por el panel de PythonAnywhere
Cotejamiento (Collation)	utf8mb4_general_ci para conservar compatibilidad exacta con el origen
Conectividad	Configuración de las credenciales en settings.py de Django mediante variables de entorno

Ventajas y desventajas:

Ventajas	Desventajas y riesgos
PythonAnywhere está especializado en aplicaciones Python y Django, lo que simplifica el despliegue.	El plan gratuito de PythonAnywhere no incluye MySQL en cuentas nuevas; se requiere un plan de pago o un complemento de base de datos.
Mantiene el mismo motor MySQL, lo que convierte la migración en homogénea.	El entorno gratuito puede presentar límites de recursos (CPU, almacenamiento, tiempo de ejecución).
Ofrece integración directa con el ORM de Django.	La base de datos gratuita administrada por PythonAnywhere no existe; es necesario confirmar las condiciones del plan.

Permite administrar la base de datos desde el panel web y desde Workbench (según las políticas del proveedor).	La conexión externa a MySQL puede estar restringida por seguridad; se deben usar mecanismos autorizados por el proveedor.
Conserva el cotejamiento utf8mb4_general_ci, evitando la pérdida de caracteres especiales.	Requiere mantener un respaldo local actualizado como medida de reversión.

Tipo de Migración

Tipo: Migración homogénea.

Justificación: el motor de base de datos de origen (MySQL, provisto por XAMPP) y el motor de destino propuesto (MySQL en PythonAnywhere) pertenecen a la misma familia de gestores. Al conservar el mismo SGBD, no es necesario convertir tipos de datos, ni adaptar sintaxis SQL, ni modificar consultas del ORM de Django. El cambio de ubicación (de local a nube) no altera por sí mismo la naturaleza del motor, por lo que la migración se clasifica como homogénea.

Si el equipo decidiera migrar a SQL Server, PostgreSQL o SQLite (motor por defecto del plan gratuito de PythonAnywhere), la migración pasaría a ser heterogénea y requeriría un análisis independiente de compatibilidad de tipos, restricciones, consultas y del controlador de Django.

Metodología de la Migración de Datos

Planeación

En esta fase se define la hoja de ruta técnica y el control de riesgos para migrar la base de datos de OperPan desde el entorno local (127.0.0.1) hacia PythonAnywhere. Mediante la estrategia Big Bang, se establecen los requisitos de conexión, la validación de datos y las pruebas necesarias para asegurar una transición fluida y sin pérdida de información.

Actividad	Aplicación en OperPan	Evidencia esperada
Definir alcance y fases	Confirmar que se migrará la base de datos completa (13 tablas) y que la aplicación Django se desplegará en PythonAnywhere.	Alcance aprobado y diagrama de arquitectura.
Identificar requerimientos	Confirmar motor y versión (MySQL), conectividad desde Django, variables de entorno y credenciales de la base de datos en PythonAnywhere.	Lista de requisitos técnicos.
Analizar riesgos	Evaluar pérdida de datos, incompatibilidad de codificación, límites del plan gratuito, exposición de credenciales e indisponibilidad temporal.	Matriz de riesgos y mitigaciones.

Analizar situación actual y destino	Inspeccionar tablas, relaciones, índices, tipos de datos y configuración actual de Django frente al entorno destino.	Inventario del origen y especificación del destino.
Factibilidad técnica y económica	Verificar compatibilidad de Django con MySQL en PythonAnywhere y evaluar las condiciones del plan gratuito o de pago.	Comparación de proveedores y decisión documentada.
Revisar esquema e integridad	Detectar claves foráneas, valores nulos no permitidos, duplicados y dependencias entre tablas.	Informe de revisión del esquema.
Seleccionar herramientas	Elegir la exportación/importación lógica mediante mysqldump y phpMyAdmin, apoyadas en MySQL Workbench cuando sea posible.	Procedimiento técnico aprobado.
Ejecutar pruebas previas	Ensayar la importación en una base de datos de prueba y validar las funcionalidades principales.	Resultados de prueba y correcciones.
Aprobar el plan	Revisar el plan con el equipo y el instructor antes del cambio definitivo.	Aprobación o registro de observaciones.

Explicación de los ítems de la tabla:

Alcance, estrategias y fases: define el traslado completo del esquema de OperPan (13 tablas) desde el servidor local hacia PythonAnywhere en una sola ejecución (Big Bang).

Identificar requerimientos: determina las credenciales de la base de datos, permisos del usuario MySQL, la conectividad desde Django y la actualización del archivo settings.py.

Análisis de riesgos: evalúa y mitiga posibles fallos como caídas de red, límites de subida de archivos en phpMyAdmin e inconsistencias en la codificación de caracteres.

Análisis de la situación actual y final: compara el entorno de desarrollo local (127.0.0.1) con la infraestructura en la nube de PythonAnywhere.

Determinar factibilidad técnica y económica: confirma que el proceso es viable con herramientas nativas y de código abierto (mysqldump, phpMyAdmin y el ORM de Django).

Análisis de integridad: garantiza que las llaves foráneas mantengan vinculadas correctamente las tablas maestras (usuarios) con las dependientes (asistencia, tareas, novedades).

Análisis de consistencia: valida que la información respete las reglas de negocio (estados válidos de usuarios, asistencia, tareas y novedades).

Análisis de valores nulos y tipos de datos: revisa restricciones NOT NULL, formatos de correo y compatibilidad con la codificación utf8mb4.

Verificar el tamaño de los objetos: mide el peso del archivo .sql exportado para evitar bloqueos por límites de carga web.

Verificar las características de la base de datos: comprueba la compatibilidad entre la versión de MySQL local y el servicio MySQL ofrecido por PythonAnywhere.

Evaluación de alternativas: compara scripts en Python, importación manual y comandos nativos del motor.

Selección de la aplicación para migrar: se elige el uso de mysqldump para la extracción y phpMyAdmin para la carga, sin desarrollos adicionales.

Pruebas del proceso: realiza un simulacro de importación en el servidor remoto antes del cambio definitivo.

Pruebas de las aplicaciones: conecta el backend de Django al servidor de PythonAnywhere y verifica las operaciones CRUD.

Migración

La migración se ejecutará durante una ventana acordada en la que no se realicen cambios en la base de datos local. Se debe conservar una copia de seguridad original y evitar que ambos entornos reciban escrituras simultáneas durante el cambio.

Fase	Actividad	Resultado esperado
Respaldo de la base de datos origen	Exportación íntegra de la base de datos local (operpan) a un archivo .sql mediante phpMyAdmin o mysqldump, asegurando la codificación utf8mb4.	Archivo SQL de respaldo.

Extracción, transformación y carga (ETL)	Creación de la base de datos y usuario en PythonAnywhere, verificación previa de compatibilidad estructural e importación del archivo .sql.	Esquema y datos cargados en el destino.
Respaldo de la base de datos destino	Generación de una copia de seguridad desde el panel de PythonAnywhere para consolidar un punto de restauración verificado.	Copia de seguridad del entorno de producción.
Configuración de Django	Actualización del archivo settings.py con las credenciales del nuevo entorno mediante variables de entorno.	Aplicación conectada al destino.
Pruebas y validación	Comparación de conteos y ejecución de pruebas funcionales sobre los módulos principales de OperPan.	Acta de validación.
Habilitación del nuevo entorno	Cambio oficial del entorno de producción cuando se cumplan los criterios de aceptación.	Entorno remoto autorizado.

Explicación de los ítems de la tabla:

Respaldo de la base de datos origen: es el punto de partida. Se toma una copia exacta de la información almacenada localmente en MySQL. El propósito es generar un archivo .sql con la estructura y los registros actuales, garantizando que ningún dato se pierda antes del traslado.

Extracción, transformación y carga (ETL): es el núcleo operativo. Se prepara el servidor destino (creación de la base de datos y del usuario con privilegios mínimos) y se ejecuta la importación del respaldo.

Respaldo de la base de datos destino: medida de seguridad final. Se genera una copia de seguridad en PythonAnywhere para dejar un punto de restauración limpio y verificado.

Configuración de Django: se actualizan las credenciales de conexión en settings.py mediante variables de entorno, evitando subir secretos al repositorio.

Pruebas y validación: se comparan conteos entre origen y destino, se revisan claves foráneas y se ejecutan pruebas funcionales sobre los módulos principales.

Habilitación del nuevo entorno: se autoriza el paso a producción únicamente cuando se cumplan los criterios de aceptación definidos.

Validación de datos

Una vez finalizada la carga, se ejecutan los protocolos de verificación para comprobar la integridad de la información, el correcto funcionamiento de las relaciones y la estabilidad general del sistema antes de habilitar el acceso en producción.

Criterio	Método de comprobación	Criterio de aceptación
Cantidad de registros	Ejecutar COUNT(*) en origen y destino para cada tabla migrada.	Los conteos coinciden, salvo diferencias justificadas y documentadas.
Integridad referencial	Comprobar claves foráneas y registros huérfanos.	No existen referencias inválidas.
Tipos y valores	Revisar fechas, horas, decimales, campos nulos y codificación de texto.	Los valores se conservan sin truncamiento ni conversión inesperada.
Acceso de Django	Probar inicio de sesión y consultas desde la aplicación desplegada.	No hay errores de conexión ni de permisos.
Pruebas funcionales	Probar usuarios, perfiles, horarios, asistencia, tareas, novedades y memorandos.	Las funciones se comportan como en el entorno de referencia.
Persistencia y recuperación	Reiniciar el servicio y verificar que los datos continúan disponibles.	Los datos persisten y existe un procedimiento documentado de recuperación.

Cierre del proyecto

Actividad	Descripción	Evidencia esperada
Reporte final	Elaboración del documento que certifica el éxito de la migración.	Documento de cierre.

Resumen ejecutivo	Síntesis de los objetivos alcanzados, tiempo de ejecución y estado actual del servicio.	Resumen de una página.
Memoria técnica	Documentación detallada con credenciales actualizadas, cadenas de conexión en Django y bitácoras del proceso.	Anexo técnico.

Técnica de Migración de Datos

La técnica seleccionada para OperPan es la exportación e importación lógica (conocida como dump and restore). Consiste en generar un archivo .sql con la estructura y los datos de la base de datos local, trasladarlo al servidor de destino e importarlo mediante phpMyAdmin o la terminal. Es la técnica más adecuada para migraciones homogéneas de alcance académico, porque no requiere replicación ni desarrollos adicionales.

Extracción

En esta etapa se obtiene la estructura y la información de la base de datos de OperPan desde el entorno local. Se identifican las tablas, campos, tipos de datos y relaciones que serán trasladados al servidor de hosting.

El procedimiento de extracción se realizó de la siguiente manera:

1. Se verificó que la base de datos local funcionara correctamente en XAMPP.
2. Se revisaron las tablas y sus relaciones (claves primarias y foráneas).
3. Se comprobó que los datos estuvieran completos.

4. Se generó el respaldo mediante mysqldump o desde phpMyAdmin (Exportar).
5. Se verificó que el archivo .sql conservara la codificación utf8mb4.

Diccionario de Datos de la Base de Datos Origen

La base de datos operpan está compuesta por 13 tablas que pueden clasificarse en dos grupos:

Tablas maestras: contienen información principal que puede ser referenciada por otras tablas. Ejemplos: usuarios_user, usuarios_perfilempleado, asistencia_horario, auth_permission.

Tablas dependientes: necesitan información de otras tablas para funcionar. Ejemplos: usuarios_registroauditoriacuenta, usuarios_passwordresettoken, asistencia_asistencia, asistencia_descansoempleado, tareas_task, memorandos_memorando, novedades_certificado, novedades_incapacidad, novedades_permiso.

Esta clasificación es importante porque permite respetar el orden de carga y conservar las claves primarias y foráneas durante la migración. El diccionario detallado de cada tabla se encuentra en el anexo técnico del proyecto y no se reproduce aquí por motivos de extensión.

Mapeo de Campos

Como la migración es homogénea, los campos mantienen prácticamente la misma estructura entre la base de datos de origen y la de destino.

Tipo en origen (MySQL)	Tratamiento propuesto en destino (MySQL)	Validación requerida

bigint(20)	BIGINT; verificar si es autoincremental y clave primaria.	Rango, signo, valores existentes y generación de identificadores.
int(11)	INT; revisar si el ancho declarado tiene efecto en la versión destino.	Valores máximos y restricciones.
varchar(n)	VARCHAR(n) conservando la longitud.	Longitud máxima y codificación.
char(n)	CHAR(n) conservando la longitud fija.	Longitud fija y valores almacenados.
longtext	LONGTEXT.	Tamaño, caracteres y truncamiento.
tinyint(1)	TINYINT(1) (booleano compatible con Django).	Valores 0/1 y comportamiento del ORM.
datetime(6)	DATETIME(6), si la versión de destino lo admite.	Precisión temporal y zonas horarias.
date	DATE.	Fechas válidas y valores nulos.

time(6)	TIME(6), si la versión de destino lo admite.	Precisión y rango.
decimal(12,2)	DECIMAL(12,2).	Precisión, escala y redondeo.

Transformación

En una migración homogénea compatible, la transformación debe ser mínima. Las actividades previstas son:

1. Normalizar la codificación a utf8mb4 si fuera necesario.
2. Verificar fechas, horas y valores decimales.
3. Revisar campos nulos y valores por defecto.
4. Comprobar enumeraciones o restricciones.
5. Adaptar instrucciones específicas de la versión del motor si se detectan incompatibilidades.
6. Verificar que los nombres de tablas y columnas no tengan conflictos con palabras reservadas.

Toda transformación debe registrarse y probarse antes de aplicarse sobre la base de datos definitiva.

Carga

La carga consiste en importar el archivo .sql a la base de datos remota. El procedimiento es el siguiente:

1. Acceder al panel de PythonAnywhere y crear la base de datos destino.
2. Seleccionar la base de datos creada para OperPan.
3. Utilizar el archivo de respaldo generado desde la base local.
4. Realizar la importación respetando el orden de dependencias (maestras primero, dependientes después).
5. Verificar que las tablas, relaciones e índices se hayan creado correctamente.
6. Configurar la conexión de Django con la nueva base de datos.

Se recomienda realizar primero una importación de ensayo y registrar los errores. Antes de la carga definitiva, debe comprobarse que el destino esté vacío o que se haya definido cómo manejar los objetos existentes.

Validación

Para comprobar que la información fue trasladada correctamente, se compara la cantidad de registros entre origen y destino. Por ejemplo, para la tabla usuarios_user:

sql

```
-- En la base de datos local
SET @cantidad_origen = (SELECT COUNT(*) FROM usuarios_user);
SELECT @cantidad_origen AS registros_origen;

-- En la base de datos del hosting
SET @cantidad_destino = (SELECT COUNT(*) FROM usuarios_user);
SELECT @cantidad_destino AS registros_destino;
```

Se debe repetir este procedimiento para cada una de las 13 tablas y documentar los resultados en el acta de validación.

Estrategia de Migración de Datos

Para el proyecto OperPan se ha seleccionado una estrategia de corte único (Big Bang). A continuación se justifica:

Fase del proyecto: el sistema se encuentra en una etapa inicial de transición hacia producción, lo que significa que actualmente no existen datos transaccionales históricos masivos ni usuarios activos operando en vivo de forma simultánea. Esto elimina la necesidad de mantener sincronizadas dos bases de datos en paralelo.

Simplicidad y coherencia estructural: al tratarse de una estructura moderada compuesta por 13 tablas interrelacionadas con claves foráneas y restricciones, realizar la migración de todo el esquema y los datos en un solo bloque garantiza la integridad referencial y evita inconsistencias.

Ventana de mantenimiento controlada: el proceso completo exportación del archivo .sql desde el entorno local, depuración de posibles incompatibilidades e importación final en PythonAnywhere puede ejecutarse en pocos minutos dentro de una ventana de mantenimiento programada.

Riesgos controlados: se conserva un respaldo completo del entorno local antes del cambio, y existe un procedimiento de reversión documentado en caso de fallos críticos. Esta estrategia es adecuada para un proyecto académico de alcance limitado.

Lista de Referencias

Django Software Foundation. (s. f.). Databases. Django documentation.

https://docs.djangoproject.com/en/4.2/ref/databases/

Oracle. (s. f.). MySQL 8.0 reference manual. MySQL Documentation.

https://dev.mysql.com/doc/refman/8.0/en/

Oracle. (s. f.). MySQL Workbench manual. MySQL Documentation.

https://dev.mysql.com/doc/workbench/en/

PythonAnywhere. (s. f.). Databases. PythonAnywhere Help.

https://help.pythonanywhere.com/pages/Databases

PythonAnywhere. (s. f.). Free accounts features. PythonAnywhere Help.

https://help.pythonanywhere.com/pages/FreeAccountsFeatures/